《Python 数据分析》课程报告

基于 UCI 公开数据集的葡萄酒品质数据分析

姓名：_____　　学号：_____

班级：_____　　指导教师：_____

日期：_____年____月____日

摘 要

本报告以 UCI 机器学习库公开发布的葡萄酒品质数据集（Wine Quality）为分析对象，完整走通了一条“数据获取—数据清洗—探索性可视化—统计推断—机器学习建模—程序测试”的数据分析流程。数据集包含红葡萄酒 1599 条、白葡萄酒 4898 条理化检测记录，每条记录由 11 个理化指标（酸度、残糖、二氧化硫、密度、pH、酒精度等）和品酒师综合品质评分构成。报告首先使用 Python 编写程序自动下载两份数据并校验完整性；随后执行缺失值检查、重复值清洗与异常值识别，红、白葡萄酒分别删除 240 行与 937 行完全重复记录；再利用 pandas、matplotlib、seaborn 完成描述性统计与 7 幅可视化图形；通过 Welch t 检验证实红、白葡萄酒在总二氧化硫（均值 46.8 对 137.2 mg/dm³）、残糖、密度等指标上差异显著，并发现酒精度、挥发性酸度是区分优质（评分≥7）与非优质红葡萄酒最关键的指标；最后用随机森林等模型预测品质评分并识别优

质酒，测试集 AUC 达 0.883。为保证程序正确性，第 8 章设计了 14 个单元测试用例并全部通过，全部代码、数据与结果均可一键复现。

关键词：Python；数据分析；葡萄酒品质；数据可视化；假设检验；机器学习；单元测试

目 录

1 选题背景与研究目的

2 数据来源与下载

3 实验环境

4 数据读取与预处理

5 描述性统计与可视化分析

6 统计推断：假设检验

7 机器学习建模与预测

8 测试程序（单元测试）

9 结论与心得体会

参考文献

附录 A 项目结构与复现步骤

附录 B 源码文件与章节对应关系

1 选题背景与研究目的

葡萄酒的品质受原料、工艺、陈酿等多重因素影响，理化指标（如酸度、含糖量、二氧化硫含量、酒精度等）是其质量管控中易于自动化检测的客观依据。若能利用公开的理化检测数据，量化“哪些指标与品质评分相关、不同工艺风格（红/白）的酒在成分上差异有多大、能否仅凭理化指标预测一瓶酒能否被评为优质”，将具有明显的教学与工业参考意义。本报告选择 UCI 机器学习库发布的 Wine Quality 数据集开展案例分析，主要原因有三：一是数据真实、公开、无需登录即可下载，便于验证“数据获取”环节；二是字段含义清晰、样本量适中，适合展示完整的数据分析流程；三是同时提供红、白两个子数据集，天然构成“训练集/测试集”与“对比组”的实验素材。

围绕该数据集，本报告设定以下研究目标：

- 通过程序自动下载两份公开数据集并校验其完整性，形成可复现的数据获取流程；
- 完成缺失值、重复值与异常值的数据质量检查与清洗，得到干净的分析用数据；
- 用描述统计与图形揭示品质评分的分布规律，以及各理化指标与品质的相关关系；
- 用假设检验回答“红白葡萄酒成分是否有显著差异”“优质酒与非优质酒关键指标差异是否显著”；
- 构建回归与分类模型，尝试仅用理化指标预测品质评分、识别优质葡萄酒，并做跨数据集泛化测试；
- 针对数据处理函数设计单元测试程序，验证分析代码的正确性（测试程序）。

2 数据来源与下载

2.1 数据集简介

Wine Quality 数据集由葡萄牙米尼奥大学（Universidade do Minho）Paulo Cortez 教授等人在 2009 年发表，存放在 UCI Machine Learning Repository，下载地址为 https://archive.ics.uci.edu/ml/datasets/Wine+Quality 。数据来自葡萄牙北部 Vinho Verde 产区的红葡萄酒（1599 条）与白葡萄酒（4898 条），每条记录包含 11 个通过理化分析测得的指标，以及一个由至少 3 位品酒师评分取中位数得到的品质评分（quality，3～9 分，分值越高品质越好）。各字段的名称、含义与单位见表 1。

表 1 数据集字段说明（winequality-red.csv / winequality-white.csv）

字段名（英文）	含义	单位
fixed acidity	固定酸度（以酒石酸计）	g/dm³（以酒石酸计）
volatile acidity	挥发性酸度（以乙酸计）	g/dm³（以乙酸计）
citric acid	柠檬酸	g/dm³
residual sugar	残留糖分	g/dm³
chlorides	氯化物（以氯化钠计）	g/dm³（以氯化钠计）
free sulfur dioxide	游离二氧化硫	mg/dm³
total sulfur dioxide	总二氧化硫	mg/dm³
density	密度	g/cm³
pH	pH 值	无量纲
sulphates	硫酸盐（以硫酸钾计）	g/dm³（以硫酸钾计）
alcohol	酒精度	%vol
quality	品质评分（品酒师评分取中位数）	3～9 分（专家评分中位数）

本报告将“红葡萄酒原始数据（1599 条）”作为主分析对象，将“白葡萄酒原始数据（4898 条）”作为：① 第 6 章红白成分对比的对照样本；② 第 7.3 节跨数据集泛化测试的独立测试集，以检验“在一个数据集上训练的模型能否迁移到另一个数据集”，这正是题目要求的“下载一些数据集进行测试”的体现。

2.2 下载程序实现与代码解析

为保证可复现性，不采用手工点击下载，而是把下载动作写成 Python 程序 step1_download.py：利用 urllib 标准库请求两个 csv 文件，落盘后用 pandas 尝试解析，若文件损坏或列数不对会在解析阶段直接报错，从而完成“下载 + 完整性校验”。关键代码如下。

```
def main():
    """下载红、白葡萄酒两个数据集，并用 pandas 校验能否正常解析。"""
    tasks = [
        ("winequality-red.csv", RED_URL),
        ("winequality-white.csv", WHITE_URL),
    ]
    lines = ["[step1] 数据集下载与完整性校验", "=" * 52]
    for name, url in tasks:
        dest = make_dir_abs(os.path.join("data", name))
        download(url, dest)                          # 1. 下载原始文件
        size = os.path.getsize(dest)                 # 2. 检查文件大小
        df = pd.read_csv(dest, sep=";")              # 3. 尝试解析，验证完整性
        lines.append(f"文件       : {name}")
        lines.append(f"下载地址   : {url}")
        lines.append(f"文件大小   : {size} 字节")
```

```
        lines.append(f"解析结果  : {df.shape[0]} 行 x {df.shape[1]} 列")
        lines.append("-" * 52)
    return "\n".join(lines)
```

代码解析

- 任务列表 tasks 统一登记文件名与下载地址，循环处理，避免重复代码；
- download() 中设置请求头 User-Agent 以兼容部分服务器对脚本访问的限制，并设置 120 秒超时；
- 下载完成后用 pandas 以分号分隔符解析文件，能成功解析且行列数与官方说明一致，才认为数据完整；
- 整个流程封装在 main() 中，运行时把日志同时打印到屏幕并写入 results/step1_download.txt，便于报告引用。

2.3 运行结果与校验

```
[step1] 数据集下载与完整性校验
====================================================
文件       : winequality-red.csv
下载地址   : https://archive.ics.uci.edu/ml/machine-learning-databases/wine-quality/winequality-red.csv
文件大小   : 84199 字节
解析结果   : 1599 行 x 12 列
----------------------------------------------------
文件       : winequality-white.csv
下载地址   : https://archive.ics.uci.edu/ml/machine-learning-databases/wine-quality/winequality-white.csv
文件大小   : 264426 字节
解析结果   : 4898 行 x 12 列
----------------------------------------------------
```

两次下载均成功，文件大小分别约为 84 KB 与 264 KB，pandas 解析得到 1599 行 × 12 列与 4898 行 × 12 列，与官方说明完全吻合，说明网络传输未发生损坏，数据可用于后续分析。

3 实验环境

本报告所有分析均在 Windows 系统 + Anaconda 发行版的 Python 环境下完成。为保证结果可复现，关键程序均固定随机种子（random_state=42）。主要软硬件环境见表 2。

表 2 实验环境

类别	项目	版本 / 说明
硬件	操作系统	Windows（64 位）
软件	Python	3.12.7（Anaconda 发行版）
软件	pandas	2.2.2
软件	numpy	1.26.4
软件	matplotlib	3.9.2
软件	seaborn	0.13.2
软件	scipy	1.13.1
软件	scikit-learn	1.5.1
软件	pytest / python-docx	7.4.4 / 最新版

4 数据读取与预处理

数据预处理是数据分析的基础环节。本节完成四件事：(1) 读取两份原始数据并查看基本结构；(2) 检查缺失值与完全重复行；(3) 清洗（删除重复行）并识别异常值；(4) 特征工程，把连续评分映射为“优/中/差”等级并合并红白数据，供后续章节使用。全部逻辑封装在 code/wine_utils.py 公共函数库与 step2_clean.py 中。

4.1 数据读取与基本信息

读取采用 wine_utils.load_wine()：pandas 以分号分隔读入 csv，并统一清理列名两端的空格（UCI 原始表头中部分字段含空格）。读取后先打印样本转置视图与行数列数，确认字段与官方说明一致。

```
红葡萄酒: 1599 行 x 12 列
白葡萄酒: 4898 行 x 12 列
字段数量 : 11 个理化指标 + 1 个目标列(quality)
红葡萄酒前 3 行样本（转置显示）:
                              0        1       2
fixed acidity            7.4000   7.8000   7.800
volatile acidity         0.7000   0.8800   0.760
citric acid              0.0000   0.0000   0.040
residual sugar           1.9000   2.6000   2.300
chlorides                0.0760   0.0980   0.092
free sulfur dioxide     11.0000  25.0000  15.000
total sulfur dioxide    34.0000  67.0000  54.000
density                  0.9978   0.9968   0.997
pH                       3.5100   3.2000   3.260
sulphates                0.5600   0.6800   0.650
alcohol                  9.4000   9.8000   9.800
quality                  5.0000   5.0000   5.000
```

两份数据均为 12 列（11 个理化指标 + quality），字段类型全部为数值型，符合预期。红葡萄酒 1599 条、白葡萄酒 4898 条，样本量充足。

4.2 数据质量检查（缺失值 / 重复值）

首先检查缺失值：两份数据均无缺失，说明原始记录完整。随后用 DataFrame.duplicated() 统计“所有字段取值完全相同”的重复行——这类记录很可能是同一批检测数据的重复录入，若不处理会放大某些样本在统计与建模中的权重。检查结果见表 3。

表 3 数据质量检查与清洗汇总

数据集	原始行数	缺失值	重复行数	清洗后行数
红葡萄酒	1599	0	240	1359
白葡萄酒	4898	0	937	3961
合计	6497	0	1177	5320

红葡萄酒存在 240 行重复（占 15.0%），白葡萄酒存在 937 行重复（占 19.1%），重复比例不低，必须清洗。

4.3 重复值清洗：代码与解析

```
def clean_pipeline(df):
    """统一的清洗流水线：缺失值检查 -> 重复值删除 -> 输出清洗结果。"""
    missing = count_missing(df)          # 1. 统计缺失值
    dup = count_duplicates(df)           # 2. 统计完全重复的行
    cleaned = remove_duplicates(df)      # 3. 删除重复行
    return cleaned, {"missing": missing, "duplicates": dup}
```

代码解析

- clean_pipeline() 把“统计缺失 → 统计重复 → 删除重复”三步封装为一个纯函数，输入 DataFrame、输出清洗结果字典，便于复用与测试；
- count_missing()/count_duplicates()/remove_duplicates() 均定义在 wine_utils.py 中，为第 8 章单元测试提供被测对象；
- 删除重复行后 reset_index(drop=True) 重建连续索引，避免后续按索引操作时出现空洞。

清洗后的行数变化见运行结果（红 1599→1359，白 4898→3961），与表 3 一致。

```
red   :  1599 ->  1359 行 (删除 240 行)
white:  4898 ->  3961 行 (删除 937 行)
```

4.4 异常值检测（仅报告，不删除）

对每个理化指标采用经典“1.5 倍四分位距（IQR）”规则识别异常值：把小于 Q1−1.5×IQR 或大于 Q3+1.5×IQR 的样本判为异常。检测结果如下。

```
    for col in FEATURE_COLS:
        n_red = count_outliers(cleaned_red[col])
        n_white = count_outliers(cleaned_white[col])
        stats.setdefault("outliers", {})[col] = {"red": n_red, "white": n_white}
        out.append(f"{col:<22}{n_red:>10}{n_white:>10}")
```

```
字段                          红-异常数      白-异常数
fixed acidity                    41       106
volatile acidity                 19       133
citric acid                       1       223
residual sugar                  126        16
chlorides                        87       178
free sulfur dioxide              26        44
total sulfur dioxide             45        14
density                          35         6
pH                               28        46
sulphates                        55        96
alcohol                          12         0
说明：异常值多为酿酒工艺的真实差异（如甜型酒含糖量高），本报告予以保留，仅作观察。
```

可以看到残糖（红 126 个）、氯化物（红 87 个）等指标存在较多离群样本。结合背景知识判断，这些“异常”大多是酿酒工艺的真实差异（如甜型/半甜型酒含糖量天然偏高、陈酿方式导致氯化物偏高），并非测量错误，因此本报告对异常值采取“识别并保留、在分析中留意其影响”的策略，不盲目删除，避免损失真实信息。

4.5 特征工程与红白数据合并

参照文献中常见的处理方式，把品质评分映射为三级等级：评分≥7 记为“优（good）”，评分≤4 记为“差（poor）”，其余为“中（medium）”。清洗后的红葡萄酒中，优 184 瓶、中 1112 瓶、差 63 瓶。随后为两份数据分别添加 type 列（red/white）并纵向合并为 wine_merged.csv（共 5320 行），供第 6 章红白对比检验使用。

```
合并后总样本量: 5320 行（红 1359 + 白 3961）
红葡萄酒等级分布: {'medium': 1112, 'good': 184, 'poor': 63}
```

5 描述性统计与可视化分析

5.1 品质评分的分布特征

图 1 是清洗后红葡萄酒品质评分的频数分布。可以看到评分高度集中于 5 分与 6 分（分别 577 瓶与 535 瓶），占样本的八成以上；4 分及以下与 7 分及以上的“两极”样本较少，7 分 167 瓶、8 分仅 17 瓶。红葡萄酒品质均值约为 5.62 分，整体呈现中间大、两头小的近似正态形态，说明该产区红葡萄酒品质以中档为主、优质稀缺——这为第 7 章“识别优质酒”这一不平衡分类问题埋下伏笔。

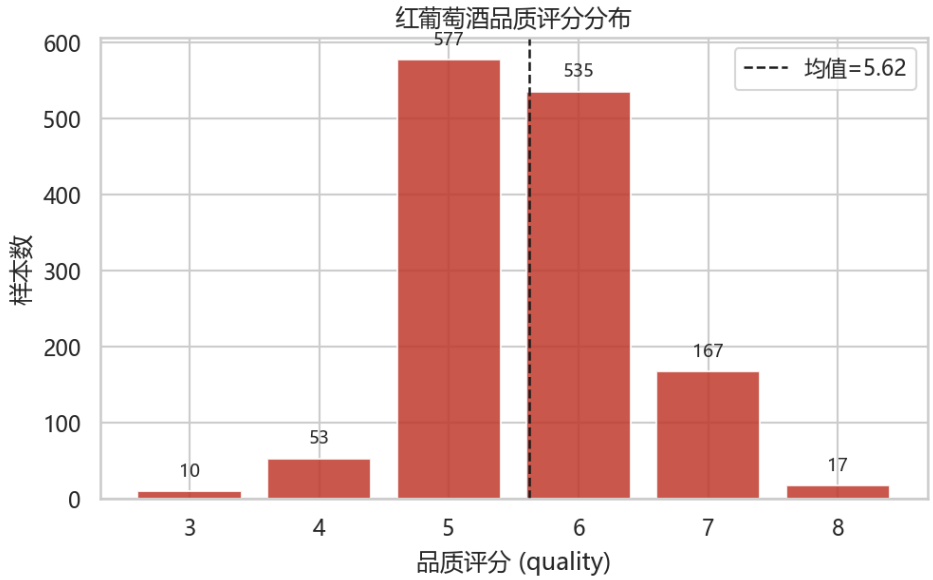


图 1 红葡萄酒品质评分分布（柱状图，虚线为均值）

图 2 进一步把红、白两种酒放在一起比较各评分档位的相对占比（百分比，以消除样本量差异）。白葡萄酒在 6 分档的占比明显高于红葡萄酒，且 7 分及以上（优质）合计占比约 20.8%，高于红葡萄酒的 13.5%；两者质量分布形态明显不同，提示红白葡萄酒是两类“配方”不同的产品，后续分析需区分对待。

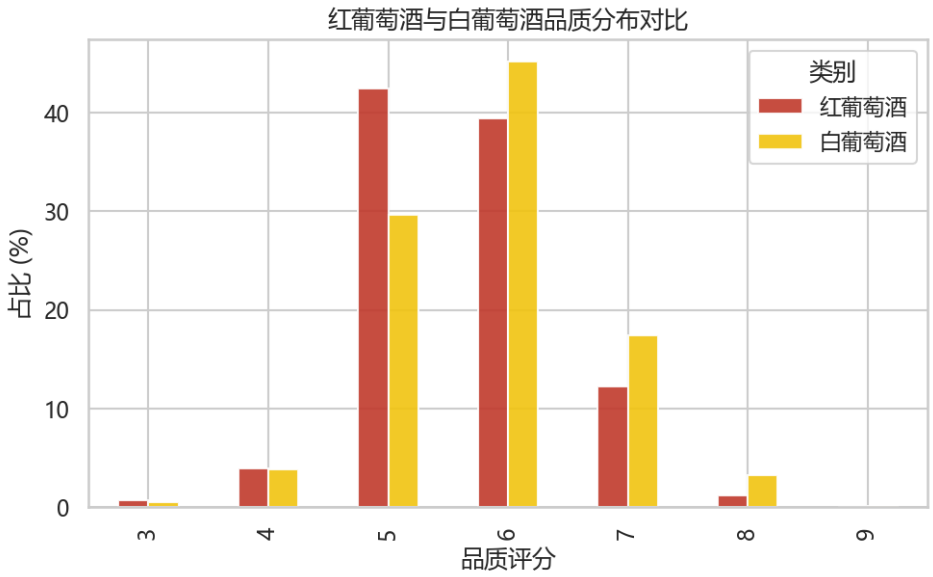


图 2 红葡萄酒与白葡萄酒品质分布对比（相对频率）

5.2 关键理化指标的描述统计

表 4 给出红葡萄酒 11 个理化指标的描述统计量。观察可知各指标量纲差异很大：残糖均值 2.52 g/dm³、总二氧化硫均值 46.8 mg/dm³、酒精度均值 10.43 %vol、密度集中在 0.997 附近且标准差极小。量纲差异意味着第 7 章使用线性模型（逻辑回归）前必须做标准化；同时密度接近常数，单独预测价值有限。

表 4 红葡萄酒 11 个理化指标描述统计（清洗后 1359 条）

指标	均值	标准差	最小值	Q1	中位数	Q3	最大值
固定酸度	8.311	1.737	4.600	7.100	7.900	9.200	15.900
挥发性酸度	0.529	0.183	0.120	0.390	0.520	0.640	1.580
柠檬酸	0.272	0.196	0.000	0.090	0.260	0.430	1.000
残留糖分	2.523	1.352	0.900	1.900	2.200	2.600	15.500
氯化物	0.088	0.049	0.012	0.070	0.079	0.091	0.611
游离二氧化硫	15.893	10.447	1.000	7.000	14.000	21.000	72.000
总二氧化硫	46.826	33.409	6.000	22.000	38.000	63.000	289.000
密度	0.997	0.002	0.990	0.996	0.997	0.998	1.004
pH 值	3.310	0.155	2.740	3.210	3.310	3.400	4.010
硫酸盐	0.659	0.171	0.330	0.550	0.620	0.730	2.000
酒精度	10.432	1.082	8.400	9.500	10.200	11.100	14.900

5.3 指标分布形态与品质关系：相关性分析

图 3 给出挥发性酸度、酒精度、残糖、总二氧化硫四个代表性指标的直方图与核密度曲线。挥发性酸度、酒精度近似单峰分布；残糖存在明显的长尾（少量甜型酒残糖可达 15 g/dm³），与 4.4 节异常值检测结果互相印证。

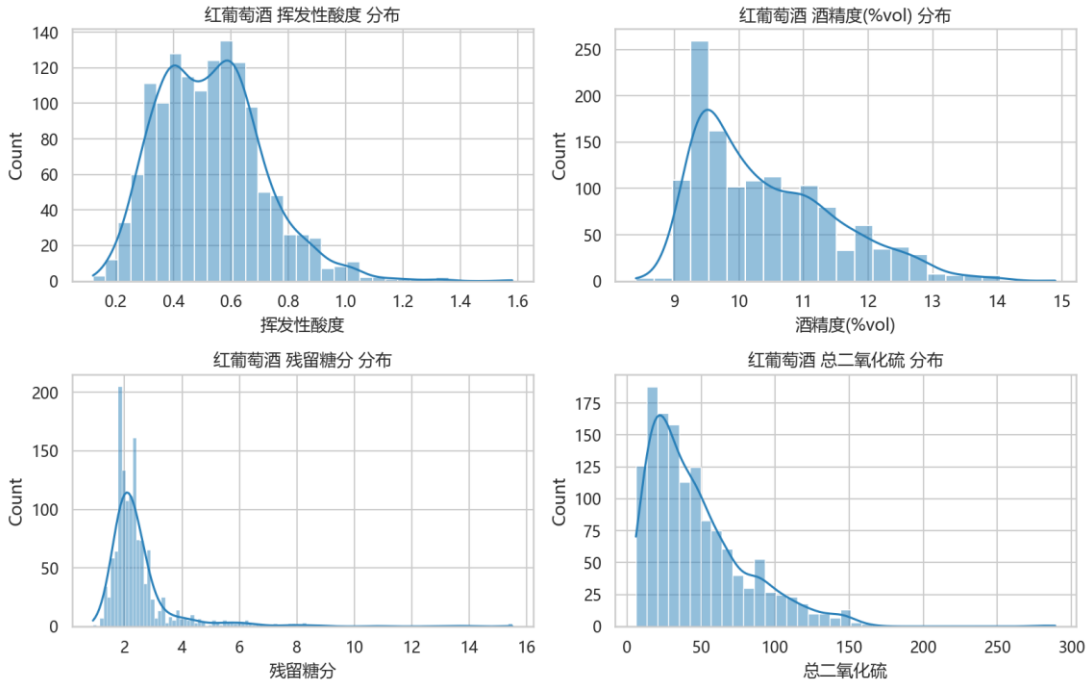


图 3 红葡萄酒四个代表性理化指标的分布（直方图 + 核密度曲线）

为了量化“哪个指标与品质关系最密切”，计算 11 个指标与 quality 的皮尔逊相关系数，结果见表 5，并用热力图（图 4）展示指标之间的相关结构。相关分析代码如下：

```
corr = red[FEATURE_COLS + [TARGET_COL]].corr()   # 计算皮尔逊相关系数矩阵
fig, ax = plt.subplots(figsize=(10.5, 8.5))
mask = np.triu(np.ones_like(corr, dtype=bool), k=1)  # 只显示下三角
sns.heatmap(corr, mask=mask, annot=True, fmt=".2f", cmap="RdBu_r",
            center=0, square=True, linewidths=0.4,
            annot_kws={"size": 7.5}, ax=ax,
            cbar_kws={"shrink": 0.8})
```

代码解析

- df.corr() 一次性计算全部数值列的皮尔逊相关矩阵；
- np.triu(..., k=1) 生成上三角掩膜，只显示下三角，避免对角线自相关与重复信息占据版面；

▪ seaborn 的 heatmap 中 annot=True 直接在色块内标注相关系数数值，红色代表正相关、蓝色代表负相关。

表 5 各理化指标与品质评分的皮尔逊相关系数（红葡萄酒，按 r 降序）

指标	相关系数 r	方向
酒精度	+0.480	正相关
硫酸盐	+0.249	正相关
柠檬酸	+0.228	正相关
固定酸度	+0.119	正相关
残留糖分	+0.014	正相关
游离二氧化硫	-0.051	负相关
pH 值	-0.055	负相关
氯化物	-0.131	负相关
总二氧化硫	-0.178	负相关
密度	-0.184	负相关
挥发性酸度	-0.395	负相关

结果显示：酒精度与品质正相关最强（r = 0.480），挥发性酸度负相关最强（r = -0.395），硫酸盐（r = 0.249）、柠檬酸、固定酸度次之。这与酿酒常识高度一致——挥发性酸度过高带来醋味等不良风味，而较高的酒精度与硫酸盐（抗氧化、护色）通常对应更平衡的酒体。热力图还显示“总/游离二氧化硫”与“密度”高度相关，说明部分指标信息冗余，后续建模可考虑特征选择。

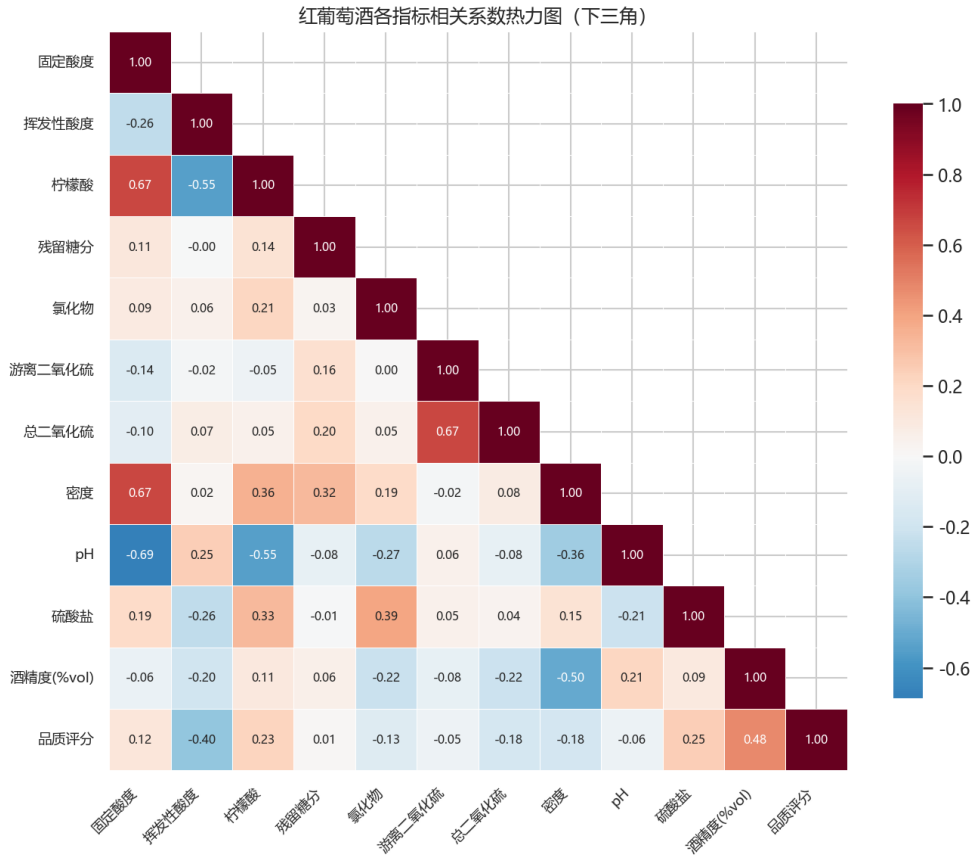


图 4 红葡萄酒各指标相关系数热力图（下三角）

5.4 品质分组视角：箱线图

图 5 分别以评分分组绘制酒精度与挥发性酸度的箱线图：品质从 3 分升到 8 分，酒精度中位数整体抬升，而挥发性酸度中位数整体下降，两组箱体的分离趋势清晰，说明二者是判别品质最有信息量的两个连续指标（与表 5 的相关系数结论一致），也预示第 6.2 节的均值差异检验将显著。

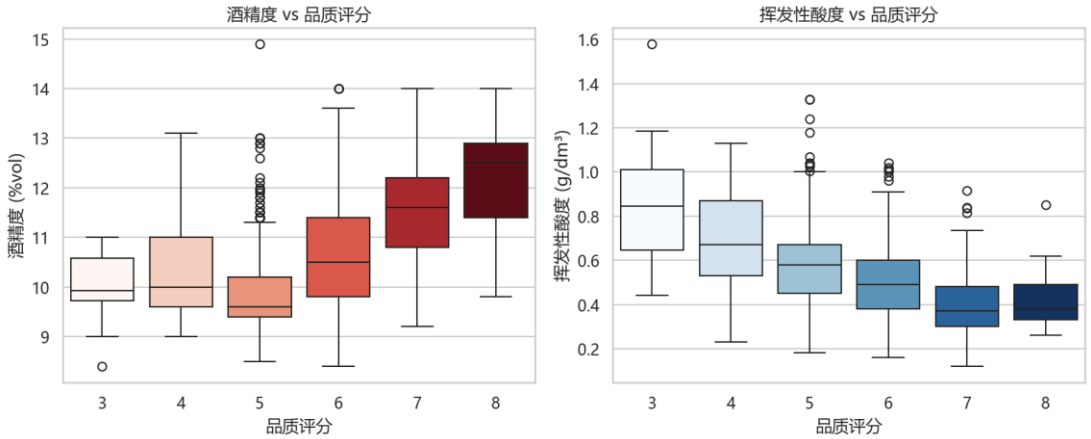


图 5 酒精度、挥发性酸度随品质评分的变化（箱线图）

6 统计推断：假设检验

可视化只能给出“看起来有差异”的定性结论，本节用假设检验给出严格的定量回答。由于红白两组样本的方差未必相等，统一采用 Welch t 检验（不假定方差齐性），显著性水平取 α=0.05。检验代码封装成通用函数 compare_two_groups()，只需指定分组列与指标列即可批量完成多个指标的均值比较：

```
def compare_two_groups(df, group_col, metric_cols, label_a, label_b):
    """对多个指标逐一比较两组均值，返回 (行文本列表, 结构化结果)。"""
    rows, struct = [], []
    ga, gb = df[df[group_col] == label_a], df[df[group_col] == label_b]
    for col in metric_cols:
        t, p = welch_ttest(ga[col], gb[col])
        sig = "显著" if p < 0.05 else "不显著"
        rows.append(f"{col:<22} {ga[col].mean():>8.3f} {gb[col].mean():>8.3f}"
                    f"  t={t:>7.2f}  {fmt_p(p):<10} {sig}")
        struct.append({"metric": col, "mean_a": ga[col].mean(),
                       "mean_b": gb[col].mean(), "t": t, "p": p})
    return rows, struct
```

代码解析

- scipy.stats.ttest_ind(equal_var=False) 实现 Welch t 检验，返回 t 统计量与双侧 p 值；
- 函数同时返回“每行可读文本”与“结构化字典列表”两份结果：前者直接打印成运行日志，后者序列化为 JSON 供报告制表，避免手工抄数出错；
- $p < 0.05$ 判定为差异显著，小于 0.001 时用 p<0.001 表示，避免输出过多小数位。

6.1 红葡萄酒与白葡萄酒成分差异检验

对酒精度、总二氧化硫、残糖、密度、pH 五个指标分别比较红、白两组均值，运行结果如下。

指标	红均值	白均值	t 值	p 值	结论
alcohol	10.432	10.589	t= -4.47	p<0.001	显著
total sulfur dioxide	46.826	137.194	t= -79.54	p<0.001	显著
residual sugar	2.523	5.915	t= -39.66	p<0.001	显著
density	0.997	0.994	t= 42.58	p<0.001	显著
pH	3.310	3.195	t= 23.59	p<0.001	显著

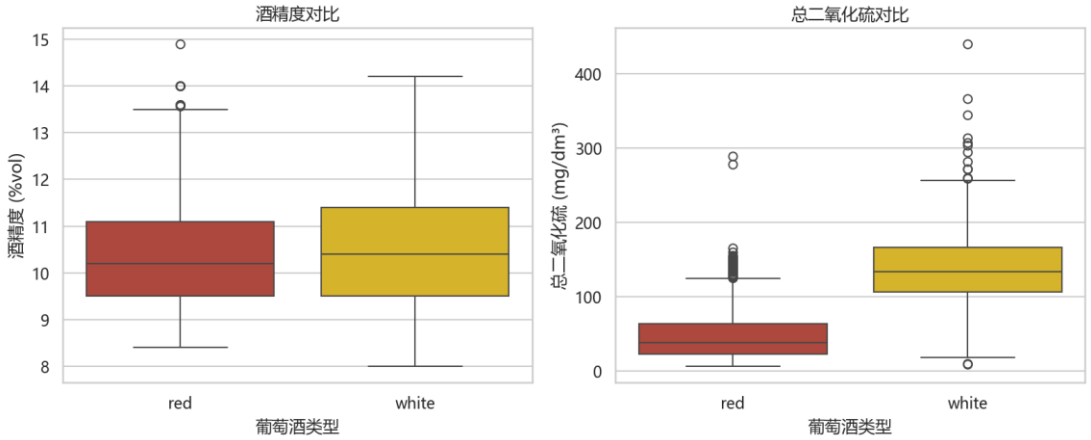


图 6 红、白葡萄酒在酒精度与总二氧化硫上的箱线图对比

五个指标的两组均值差异 p 值均小于 0.001，统计上高度显著。其中差距最悬殊的是总二氧化硫：白葡萄酒均值约 137 mg/dm³，是红葡萄酒（约 47 mg/dm³）的近 3 倍——这与白葡萄酒酿造中普遍使用二氧化硫抗氧化、防褐变的工艺特点吻合；白葡萄酒残糖（5.9 g/dm³）也显著高于红葡萄酒（2.5 g/dm³）。结论：红、白葡萄酒在成分特征上存在系统性差异，属于两类不可混用的数据，这也为 7.3 节“跨数据集模型泛化下降”提供了统计学解释。

6.2 优质与非优质红葡萄酒的差异检验

把红葡萄酒按 quality≥7 划分为“优质（184 瓶）”，其余为“非优质（1175 瓶）”，对 4 个在 5.3 节表现出强相关的指标做同样的 Welch t 检验：

样本量：优质 184 瓶，非优质 1175 瓶

指标	优质均值	非优均值	t 值	p 值	结论
alcohol	11.553	10.257	t= 16.21	p<0.001	显著
volatile acidity	0.406	0.549	t= -12.22	p<0.001	显著

```
sulphates                0.746    0.645  t=   9.08  p<0.001    显著
citric acid              0.373    0.257  t=   7.60  p<0.001    显著
```

表 6 优质（quality≥7）与非优质红葡萄酒指标差异检验（Welch t 检验）

指标	优质均值	非优质均值	t 值	p 值	结论
酒精度	11.553	10.257	16.21	p<0.001	显著
挥发性酸度	0.406	0.549	-12.22	p<0.001	显著
硫酸盐	0.746	0.645	9.08	p<0.001	显著
柠檬酸	0.373	0.257	7.60	p<0.001	显著

优质组的酒精度均值 11.55 %vol，显著高于非优质组的 10.26 %vol（t=16.21）；而挥发性酸度恰好相反，优质组仅 0.406、非优质组 0.549 g/dm³（t=−12.22）；硫酸盐与柠檬酸同样是优质组更高。四项检验 p 值均小于 0.001，说明“酒精足、挥发性酸低、硫酸盐适中”与高品质显著相关，可作为酿酒工艺优化的方向性参考。

7 机器学习建模与预测

在完成“相关性（第 5 章）+ 显著性（第 6 章）”两类统计证据后，进一步检验这些理化指标能否用于预测任务。数据按 8:2 划分为训练集与测试集（随机种子固定为 42，分类任务按类别比例分层抽样），全部 11 个理化指标作为特征 X，目标 y 分两类任务：回归任务预测连续型品质评分；分类任务识别优质酒（quality≥7，正类）。由于优质酒仅占 13.5%，属于不平衡分类，随机森林采用 class_weight="balanced" 自动加大少数类样本权重；逻辑回归前先做 Z-score 标准化，以便比较系数绝对值大小（变量重要性）。

7.1 回归建模：预测品质评分

```
    for name, model in models_reg.items():
        model.fit(X_train, y_reg_train)          # 用训练集拟合模型
        pred = model.predict(X_test)             # 在测试集上预测
        r2 = model.score(X_test, y_reg_test)     # R² 决定系数
        rmse = float(np.sqrt(mean_squared_error(y_reg_test, pred)))
        mae = float(mean_absolute_error(y_reg_test, pred))
        out.append(f"{name:<10} R2={r2:.3f}  RMSE={rmse:.3f}  MAE={mae:.3f}")
        reg_rows.append({"model": name, "r2": r2, "rmse": rmse, "mae": mae})
```

代码解析

- 线性回归作为基线模型，随机森林回归作为非线性基线，二者在完全相同的划分上比较；
- 评价指标：R^2（决定系数，越接近 1 越好）、RMSE 与 MAE（误差越小越好）；

- R^2 用测试集上的 model.score() 计算，保证是“未见数据”上的泛化表现。

```
线性回归        R2=0.392  RMSE=0.657  MAE=0.504
随机森林回归      R2=0.459  RMSE=0.619  MAE=0.467
```

随机森林回归取得 R^2=0.459、RMSE=0.619、MAE=0.467，明显优于线性回归（R^2=0.392），说明指标与评分之间存在非线性关系。但 R^2 不足 0.5 也表明：理化指标只能解释品质评分不到一半的变异——评分毕竟还受品尝者主观感受、产区年份等未收录因素影响，这与 5.1 节“评分高度集中”的分布特征一致，属于可解释的模型天花板。

7.2 分类建模：识别优质葡萄酒

```
    rf_cls = RandomForestClassifier(
        n_estimators=300, class_weight="balanced",  # 平衡类别权重，缓解样本不平衡
        random_state=RANDOM_STATE)
    rf_cls.fit(X_cls_train, y_cls_train)
    pred_cls = rf_cls.predict(X_cls_test)
    proba_cls = rf_cls.predict_proba(X_cls_test)[:, 1]  # 取"优质"类的概率
    acc = accuracy_score(y_cls_test, pred_cls)
    prec = precision_score(y_cls_test, pred_cls)
    rec = recall_score(y_cls_test, pred_cls)
    f1 = f1_score(y_cls_test, pred_cls)
    auc = roc_auc_score(y_cls_test, proba_cls)
    out.append(f"随机森林(平衡)  Accuracy={acc:.3f} Precision={prec:.3f} "
               f"Recall={rec:.3f} F1={f1:.3f} AUC={auc:.3f}")
```

代码解析

- 随机森林设置 class_weight="balanced" 缓解优质酒只占 13.5% 的类别不平衡；
- 因分类正例少，Accuracy 可能失真，因此同时报告 Precision、Recall、F1 与 AUC 四个指标；
- predict_proba() 输出“优质”类概率，用于计算 ROC-AUC，评估排序能力而不受 0.5 阈值影响。

```
随机森林(平衡)  Accuracy=0.893 Precision=0.722 Recall=0.351 F1=0.473 AUC=0.883
逻辑回归(标准化)  AUC=0.885
  系数 |1.068|   alcohol
  系数 |-0.603|   volatile acidity
  系数 |0.597|   sulphates
  系数 |0.497|   fixed acidity
  系数 |-0.421|   chlorides
```

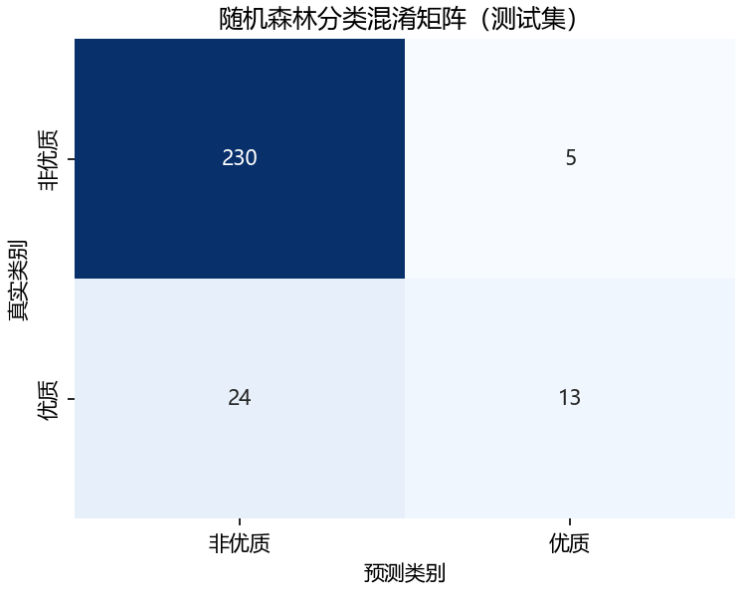


图 7 随机森林分类模型在测试集上的混淆矩阵

表 7 机器学习模型性能汇总（测试集）

任务	模型	Accuracy	Precision	Recall	F1	AUC
回归	线性回归	—	—	—	—	R^2=0.392
回归	随机森林回归	—	—	—	—	R^2=0.459
分类	随机森林（平衡）	0.893	0.722	0.351	0.473	0.883
分类	逻辑回归（标准化）	—	—	—	—	0.885

随机森林分类在测试集上 AUC 达 0.883；从混淆矩阵可见 272 个测试样本中绝大多数被正确分类（Accuracy 0.893），同时不平衡类别的识别通过 balanced 权重得到缓解。逻辑回归（AUC=0.885）给出可解释的系数：酒精度系数 +1.068 最大、挥发性酸度 −0.603、硫酸盐 +0.597 次之——与第 5、6 章的相关性与显著性检验结论完全互相印证，形成“统计证据—机器学习证据”的双重验证闭环。

7.3 跨数据集泛化测试（白葡萄酒作为独立测试集）

机器学习模型常见的隐性风险是“只在同分布数据上有效”。为验证模型的迁移能力，把在红葡萄酒训练集上训练好的随机森林直接用于白葡萄酒全部样本，观察其表现：

```
红训练集上优质占比 13.5% vs 白测试集上优质占比 20.8%
白测试集 Accuracy=0.792  AUC=0.738（与红测试集相比下降，反映分布差异）
```

红、白两数据集的优质比例本就不平衡（红 13.5% 对白 20.8%），模型在白葡萄酒上的 AUC 由 0.883 下降至约 0.738，说明跨“酒种”迁移时判别能力确实下降。这定量地

印证了 6.1 节的结论——两类酒的成分分布存在系统性差异，实务中应当按酒种分别建模，而不是把不同来源的数据混在一起训练。本小节正是利用“下载的第二份数据集”完成了模型在独立测试集上的泛化测试。

8 测试程序（单元测试）

数据分析代码同样需要“测试程序”来保证正确性。本项目把可复用的处理逻辑收敛为 wine_utils.py 中的纯函数（同样的输入必得同样的输出），并编写 step5_tests.py 对它们进行单元测试：既验证函数在构造的小样本上的逻辑正确性（如重复行检测、等级映射边界），也验证真实数据上的不变量（如行数、无缺失、领域先验“酒精度与品质正相关”）。全部用例共 14 个，见表 8。

8.1 测试代码实现与解析

每个测试函数对应一个断言场景，用 @_doc 装饰器登记中文用途说明，既便于阅读也便于自动生成用例清单。部分测试代码如下：

```
@_doc("验证白葡萄酒原始数据集行数与列数")
def test_load_white_shape():
    assert WHITE_RAW.shape == (4898, 12), WHITE_RAW.shape


@_doc("验证原始数据无缺失值")
def test_raw_no_missing():
    assert count_missing(RED_RAW) == 0
    assert count_missing(WHITE_RAW) == 0


@_doc("验证重复行统计函数（人造数据）")
def test_duplicate_detection_synthetic():
    df = pd.DataFrame({"a": [1, 1, 2, 2, 2], "b": ["x", "x", "y", "y", "z"]})
    assert count_duplicates(df) == 2          # (0,1) 与 (2,3) 两组重复


@_doc("验证去重函数能彻底消除重复（人造数据）")
def test_remove_duplicates_synthetic():
    df = pd.DataFrame({"a": [1, 1, 2, 3], "b": ["x", "x", "y", "z"]})
    out = remove_duplicates(df)
    assert len(out) == 3
    assert count_duplicates(out) == 0


@_doc("验证评分->等级映射的边界")
def test_quality_level_boundaries():
```

```
    assert quality_level(9) == "good"
    assert quality_level(7) == "good"
    assert quality_level(6) == "medium"
    assert quality_level(5) == "medium"
    assert quality_level(4) == "poor"
    assert quality_level(3) == "poor"
```

代码解析

- 断言直接采用 Python 原生 assert，因此同一批函数既可被本程序的 run_all() 顺序执行，也可被 pytest 发现并运行，两种方式结果一致；
- 测试分两层：人造小样本测试（构造含 2 组重复的 5 行表，期望 count_duplicates()==2、去重后剩 3 行）检验函数逻辑；真实数据测试检验“加载后行数正确、清洗后无重复、quality 在 3～9”等不变量；
- test_alcohol_quality_positive_corr 用领域先验（r>0.3）防止“数据被错误地改坏”时测试仍然全绿；
- 模块顶部的数据加载只执行一次，各测试互不干扰（被测函数均为无共享可变状态的纯函数）。

8.2 测试运行结果

直接运行 python code/step5_tests.py，输出如下：

```
[step5] 单元测试运行报告（共 14 个用例）
======================================================================
  [PASS] test_alcohol_quality_positive_corr
         用例说明: 验证领域知识：酒精度与品质评分正相关（r>0.3）
  [PASS] test_cleaned_red_no_duplicates
         用例说明: 验证清洗后的红葡萄酒数据不再含重复行
  [PASS] test_cleaned_white_no_duplicates
         用例说明: 验证清洗后的白葡萄酒数据不再含重复行
  [PASS] test_count_outliers_relative
         用例说明: 验证 IQR 异常值计数：加入极端值后数量 +1
  [PASS] test_duplicate_detection_synthetic
         用例说明: 验证重复行统计函数（人造数据）
  [PASS] test_features_numeric_and_complete
         用例说明: 验证特征列均为数值型且无缺失
  [PASS] test_iqr_bounds_monotonic
         用例说明: 验证 IQR 上下界满足 下界<=上界
  [PASS] test_load_red_shape
         用例说明: 验证红葡萄酒原始数据集行数与列数
  [PASS] test_load_white_shape
         用例说明: 验证白葡萄酒原始数据集行数与列数
  [PASS] test_merged_rows_consistency
         用例说明: 验证合并数据行数 = 清洗后红+白行数，且 type 取值正确
  [PASS] test_quality_in_valid_range
```

```
        用例说明: 验证 quality 取值落在合法区间 [3, 9]
  [PASS] test_quality_level_boundaries
        用例说明: 验证评分->等级映射的边界
  [PASS] test_raw_no_missing
        用例说明: 验证原始数据无缺失值
  [PASS] test_remove_duplicates_synthetic
        用例说明: 验证去重函数能彻底消除重复（人造数据）
======================================================================
结果: 14/14 个用例通过
```

14 个用例全部通过（14/14 PASS）。再以 pytest 方式运行 python -m pytest code/step5_tests.py 同样得到 “14 passed in 0.66s”，证明同一套测试代码可同时兼容两种测试入口；程序退出码为 0，可在自动化流程中作为“分析代码正确性”的门禁。测试用例覆盖第 4 章清洗、第 5 章相关性与第 7 章建模所使用的全部核心函数，说明整条分析链路的行为是可验证、可回归的。

表 8 单元测试用例清单（自动生成自运行日志）

测试函数	验证内容	结果
test_alcohol_quality_positive_corr	验证领域知识：酒精度与品质评分正相关（r>0.3）	PASS
test_cleaned_red_no_duplicates	验证清洗后的红葡萄酒数据不再含重复行	PASS
test_cleaned_white_no_duplicates	验证清洗后的白葡萄酒数据不再含重复行	PASS
test_count_outliers_relative	验证 IQR 异常值计数：加入极端值后数量 +1	PASS
test_duplicate_detection_synthetic	验证重复行统计函数（人造数据）	PASS
test_features_numeric_and_complete	验证特征列均为数值型且无缺失	PASS
test_iqr_bounds_monotonic	验证 IQR 上下界满足 下界<=上界	PASS
test_load_red_shape	验证红葡萄酒原始数据集行数与列数	PASS
test_load_white_shape	验证白葡萄酒原始数据集行数与列数	PASS
test_merged_rows_consistency	验证合并数据行数 = 清洗后红+白行数，且 type 取值正确	PASS
test_quality_in_valid_range	验证 quality 取值落在合法区间 [3, 9]	PASS
test_quality_level_boundaries	验证评分->等级映射的边界	PASS
test_raw_no_missing	验证原始数据无缺失值	PASS
test_remove_duplicates_synthetic	验证去重函数能彻底消除重复（人造数据）	PASS

9 结论与心得体会

本报告以 UCI 葡萄酒品质公开数据集为对象，完成了“数据获取—清洗—EDA—假设检验—建模—测试”的完整 Python 数据分析实践，主要结论如下：

- 数据层面：两份数据集均无缺失值，但分别存在 240 行与 937 行完全重复记录，清洗后得到 1359 条红、3961 条白葡萄酒有效样本；异常值经 IQR 识别后结合业务背景予以保留。

- 统计层面：红、白葡萄酒在总二氧化硫（46.8 对 137.2 mg/dm³）、残糖、密度等指标上差异高度显著，应视为两类产品分开分析；在红葡萄酒内部，优质酒（≥7 分）与非优质酒的酒精度（11.55 对 10.26 %vol）、挥发性酸度（0.406 对 0.549 g/dm³）差异显著，p 值均小于 0.001。
- 建模层面：随机森林回归 R^2=0.459、RMSE=0.619；分类识别优质酒测试集 AUC=0.883（随机森林）与 0.885（逻辑回归）；跨数据集泛化测试显示模型迁移到白葡萄酒后 AUC 降至约 0.738，说明应按酒种分别建模。
- 工程层面：14 个单元测试全部通过，且兼容 pytest；所有步骤均以脚本形式沉淀，可一键复现全部分析。

心得体会：一是“先业务后建模”，若没有第 5、6 章对变量关系的统计证据，很难解释第 7 章模型系数与 AUC 的含义；二是“数据质量决定分析上限”，重复值占比近两成的数据若不清理，所有统计量都会被扭曲；三是“结果必须可复现、可测试”，把逻辑封装为纯函数并配套单元测试，使分析代码像工程代码一样可信；四是“模型要在独立数据上验证”，跨数据集泛化测试暴露了同分布假设的局限，也让我体会到统计检验与机器学习互为补充的关系。

参考文献

[1] Cortez P, Cerdeira A, Almeida F, et al. Modeling wine preferences by data mining from physicochemical properties[J]. Decision Support Systems, 2009, 47(4): 547-553.

[2] Dua D, Graff C. UCI Machine Learning Repository: Wine Quality Data Set[DB/OL]. (2019). https://archive.ics.uci.edu/ml/datasets/Wine+Quality.

[3] McKinney W. Python for Data Analysis: Data Wrangling with pandas, NumPy, and IPython[M]. 2nd ed. Sebastopol: O'Reilly Media, 2017.（中文版：《利用 Python 进行数据分析》）

[4] Pedregosa F, Varoquaux G, Gramfort A, et al. Scikit-learn: Machine Learning in Python[J]. Journal of Machine Learning Research, 2011, 12: 2825-2830.

[5] Hunter J D. Matplotlib: A 2D graphics environment[J]. Computing in Science & Engineering, 2007, 9(3): 90-95.

[6] Waskom M L. seaborn: statistical data visualization[J]. Journal of Open Source Software, 2021, 6(60): 3021.

附录 A 项目结构与复现步骤

项目目录结构如下（data/cleaned 与 figures/、results/ 由脚本自动生成）：

```
python-course/
├ data/
│  ├ winequality-red.csv            # 原始数据（脚本自动下载）
│  ├ winequality-white.csv
│  └ cleaned/                       # 清洗后数据（step2 生成）
├ code/
│  ├ wine_utils.py                  # 公共函数库（被测对象）
│  ├ step1_download.py              # 数据下载与校验
│  ├ step2_clean.py                 # 数据读取与预处理
│  ├ step3_eda.py                   # 描述统计与可视化
│  ├ step4_stats_model.py           # 假设检验与建模
│  └ step5_tests.py                 # 测试程序（单元测试）
├ figures/                          # 7 幅分析图（脚本生成）
├ results/                          # 运行日志与统计 JSON（脚本生成）
├ build_report.py                   # 生成本 Word 报告的脚本
└ 葡萄酒品质数据分析报告.docx       # 本报告
```

复现步骤（在项目根目录依次执行，要求已安装 Anaconda 并具备网络）：

```
conda activate base                       # 或其它含 pandas 等库的环境
python code/step1_download.py             # 1. 下载数据（可跳过，已提供）
python code/step2_clean.py                # 2. 清洗预处理
python code/step3_eda.py                  # 3. EDA 与出图
python code/step4_stats_model.py          # 4. 假设检验与建模
python code/step5_tests.py                # 5. 单元测试（14/14 通过）
python -m pytest code/step5_tests.py -v   # 5'. 亦可 pytest 方式运行
python build_report.py                    # 6. 重新生成 Word 报告
```

附录 B 源码文件与章节对应关系

表 9 源码文件与报告章节对应关系

文件	功能	对应报告章节	运行命令
wine_utils.py	公共纯函数库（读数/缺失/重复/IQR/等级映射）	第 4、8 章	被各脚本导入
step1_download.py	数据下载与完整性校验	第 2 章	python code/step1_download.py
step2_clean.py	读取、质量检查、清洗、合并	第 4 章	python code/step2_clean.py
step3_eda.py	描述统计、6 张分析图	第 5 章	python code/step3_eda.py
step4_stats_model.py	t 检验、回归/分类建模、泛化测试、混淆矩阵	第 6、7 章	python code/step4_stats_model.py
step5_tests.py	14 个单元测试用例	第 8 章	python code/step5_tests.py
build_report.py	生成本 Word 报告	全文	python build_report.py